Ports, Sockets & Routing

how one port serves 100,000 clients, and how packets find it

Watch the 3-min video

In one line: the IP finds the machine and the port finds the process; every TCP connection gets its own socket, named by a unique 5-tuple, so one port can hold thousands of connections; and routers forward packets hop by hop using longest prefix match.

1 Ports: which process on the machine

One server, one port: 443. Yet it holds **100,000 live WebSocket connections** at the same time. How? The answer is in how ports and sockets really work.

An **IP address** finds the machine. A **port** finds the process on that machine. Ports are **16-bit** numbers, from 0 to 65,535. Servers *listen* on well-known ports: **443** for HTTPS, **22** for SSH, **5432** for Postgres, **6379** for Redis (and 80 for HTTP, 53 for DNS). Clients use a random, temporary port on their side, called an **ephemeral port**, picked by the OS for each outgoing connection (on Linux, from 32768 to 60999 by default).

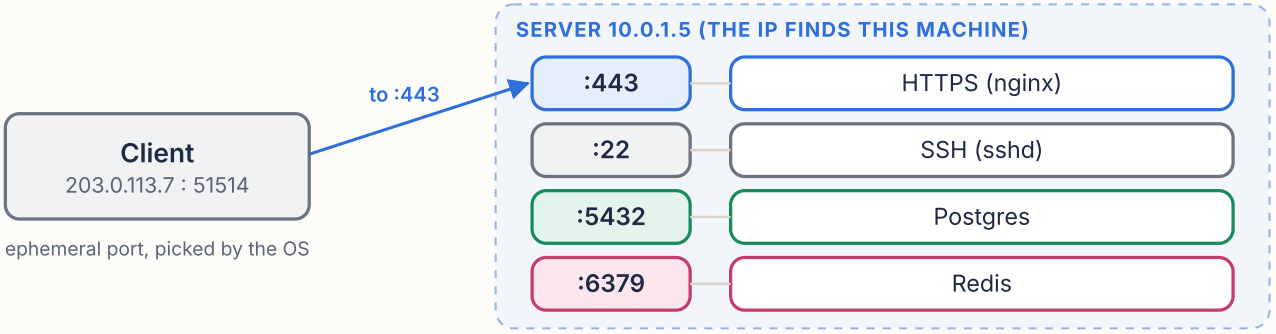


Figure 1. The client reaches 10.0.1.5 by IP; port 443 selects the web server process. The client's own port, 51514, was chosen by its OS.

2 Sockets and the 5-tuple

A **socket** is one endpoint of communication: an IP address, a port and a protocol, such as TCP. Your program asks the OS for a socket, then reads and writes through it, much like a file. On Linux it really is a file descriptor.

A TCP connection is identified by five values, the **5-tuple**:

```
( protocol, source IP, source port, destination IP, destination port )
```

Change any one of them and it is a different connection. That is why a TCP socket and a UDP socket can both use port 443 on the same machine, and why two tabs in your browser can open two separate connections to the same server: they get different ephemeral ports.

3 How one port holds 100,000 connections

The server has **one listening socket** on port 443. Each time a client connects, the OS creates a **brand new socket** for that connection and hands it to the program (the `accept()` call). All these sockets share local port 443, but each has a different client IP and port, so every 5-tuple is unique. Ports are not used up on the server side at all.

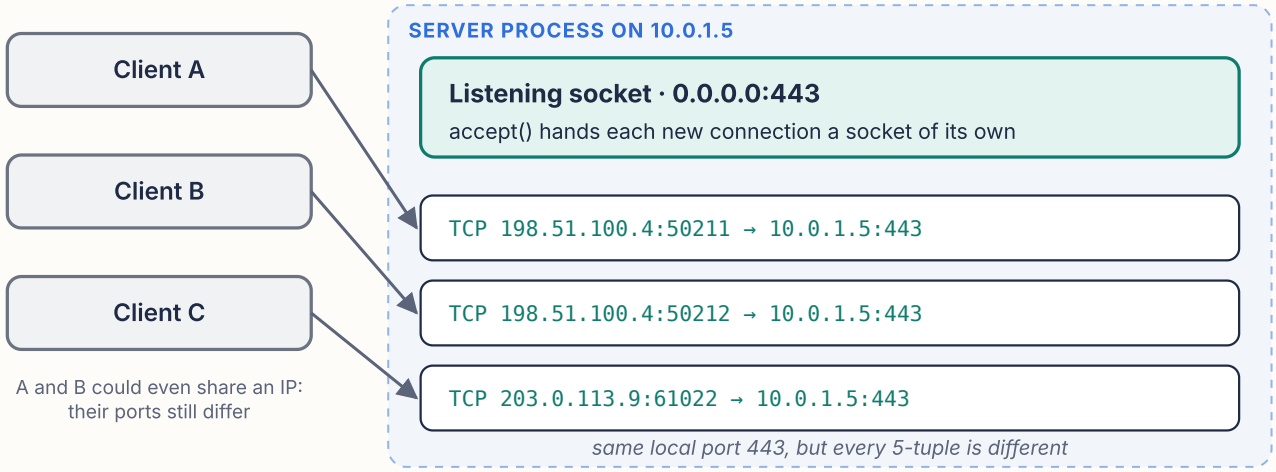


Figure 2. One listening socket on :443 accepts connections; each accepted connection becomes its own socket, told apart by the client's IP and port.

The real limits are **memory** and **file descriptors**, not ports. Every socket is a file descriptor, and many systems default to 1,024 open files per process. Every connection also costs kernel buffers plus your app's own state.

WORKED EXAMPLE: 100K WEBSOCKETS ON ONE BOX

File descriptors: 100,000 connections need 100,000 descriptors, plus some for logs and upstream connections. Raise the open-file limit (for example `ulimit -n` or systemd `LimitNOFILE`) to about 200,000.

Memory: budget per connection. If kernel buffers plus app state come to about 20 KB each, 100,000 × 20 KB ≈ **2 GB** of RAM. At 100 KB each it is 10 GB, which decides your instance size. Then load-test, because idle connections are cheap but busy ones also cost CPU.

4 Routing: how packets find the server

Every machine and router has a **route table**. For each packet, it picks the **most specific route** that matches the destination address. That rule is called **longest prefix match**: a `/24` beats a `/16`, which beats a `/0`. Addresses on the local network are delivered directly. Everything else goes to the **default gateway**, the `0.0.0.0/0` route, which matches any address.

Routers forward packets **hop by hop**. Each one looks only at the destination IP, checks its own table and passes the packet one step closer. No single router knows the full path. (Each hop also lowers the packet's TTL counter by one, so a routing loop can't circle forever; `traceroute` uses this to list the hops.)

Route table of a private app subnet

Destination	Target	Prefix bits
10.0.0.0/16	local (inside the VPC)	16
10.1.0.0/16	**peering link to VPC 2**	16
0.0.0.0/0	NAT gateway (default)	0

Packet to 10.1.4.20

matches /16 and /0: the /16 is more specific, so it goes over peering

8.8.8.8 matches only 0.0.0.0/0 → NAT gateway. 10.0.2.15 matches 10.0.0.0/16 → delivered locally. 0.0.0.0/0 matches every address, so it is the route of last resort: the default gateway.

Figure 3. Longest prefix match. The packet matches two routes; the one with more fixed bits wins.

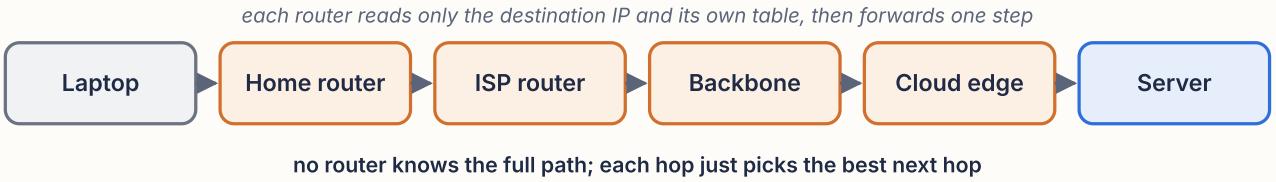


Figure 4. A request crosses several routers. Each makes a local decision; together they deliver the packet.

A **cloud VPC works the same way**. Each subnet's route table keeps local traffic (`10.0.0.0/16 → local`) inside the VPC, and sends `0.0.0.0/0` to an **internet gateway** (public subnets) or a **NAT gateway** (private subnets).

COMMON MISTAKES

- Thinking a server runs out of ports as clients connect. It doesn't: the client side picks the varying port.
- Leaving the default 1,024 open-file limit on a server meant for many connections: it fails with "too many open files".
- Sizing a WebSocket fleet by CPU only; memory per connection is usually the limit.
- Assuming the first matching route wins. The most specific one wins, whatever order the table is in.

5 Key terms

Port: 16-bit number that picks the process

5-tuple: protocol, src IP, src port, dst IP, dst port

Route table: destination prefix → next hop

Ephemeral port: temporary client-side port

Listening socket: waits on a port, accepts connections

Longest prefix match: the most specific route wins

Socket: endpoint: IP + port + protocol

File descriptor: OS handle for an open file or socket

Default gateway: the 0.0.0.0/0 route

6 Interview questions

Q1. What is a socket? BASIC

An endpoint defined by an IP address, a port and a protocol, which a program uses to send and receive data. The OS gives it to the program as a handle (a file descriptor) to read and write.

Q2. What's the difference between an IP address and a port? BASIC

The IP address finds the machine; the port (0 to 65,535) finds the process on it. 10.0.1.5:5432 is Postgres on that host, 10.0.1.5:6379 is Redis.

Q3. How can one server port accept thousands of connections? BASIC

Each connection gets its own socket, identified by the full 5-tuple. All share local port 443, but the client's IP and port make each one unique. The limits are memory and file descriptors.

Q4. You need one box to hold 100k WebSocket connections. What do you change? SCENARIO

Raise the open-file limit well above 100k (say 200k), budget memory per connection (100k × 20 KB ≈ 2 GB), tune kernel buffers if needed, and load-test. Ports are not the constraint.

Q5. A packet for 10.1.4.20 hits a table with 10.0.0.0/16 local, 10.1.0.0/16 peering, and 0.0.0.0/0 NAT. Where does it go? SCENARIO

Peering. It matches 10.1.0.0/16 and 0.0.0.0/0; longest prefix match picks the /16, the more specific route.

Q6. An app in a private subnet can't reach an external API. What do you check in routing? SCENARIO

That the subnet's route table has 0.0.0.0/0 pointing at a NAT gateway, that the NAT gateway sits in a public subnet whose 0.0.0.0/0 goes to the internet gateway, then security groups and network ACLs.

Q7. Where can ports actually run out? TRICKY

On the client side. One client IP talking to one server IP:port can only open as many connections as it has ephemeral ports, about 28,000 with Linux defaults. Proxies and load balancers hit this; fix it with more source IPs, more backend IPs, a wider port range, or connection reuse.

Q8. Do routers know the full path to a destination? TRICKY

No. Each router looks only at the destination IP, picks the best next hop from its own table and forwards the packet. The full path emerges from many local decisions, and it can change between packets.

Next: SD-B-008 · How DNS Resolution Works backendroadmap.com